

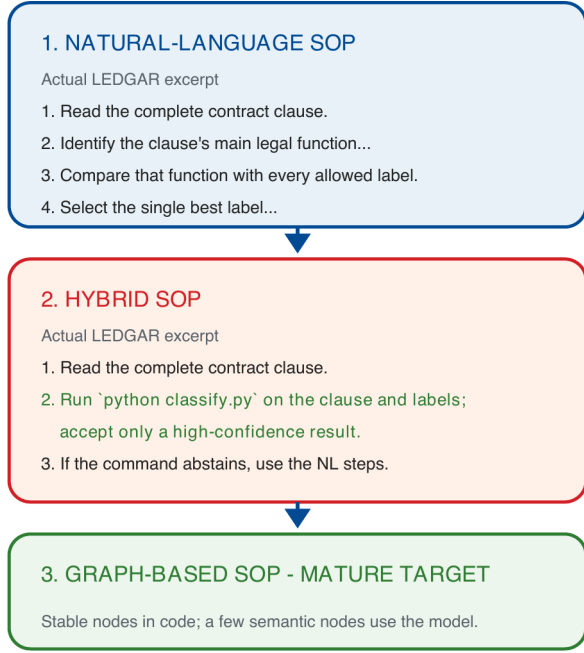


Figure 1. Actual LEDGAR SOP excerpts and the intended mature endpoint. The NL skill contains only English steps. The hybrid inserts `python classify.py` and returns to those English steps when the command abstains. The preferred mature endpoint, when validated, is a graph with only a few model-mediated nodes.

Reusable agent instructions are increasingly packaged as skills: the Agent Skills specification defines `SKILL.md` and supports references, scripts, and assets [1]; DeepAgents supports skill-based agent execution [2]; and LangGraph represents structured workflows as explicit graphs [3]. Related systems optimize language-model programs, prompts, workflows, or skills. DSPy compiles declarative model pipelines against metrics [4]; AutoFlow and AFlow generate and optimize workflows [5, 6]; Automated Design of Agentic Systems searches agent designs [7]; and SkillOpt, SkillRevise, SkillReducer, and ACES optimize or evaluate reusable agent capabilities [8-11].

PLaND addresses a narrower representation question: when can model-mediated work inside a workflow be replaced by deterministic execution under an explicit empirical contract? Its contribution is not a new general-purpose rule-mining algorithm. Rather, it combines an agent-guided candidate-evolution policy with deterministic evaluation and promotion boundaries. The current experiments test whether selected substitutions are safe and useful; they do not independently benchmark general autonomous discovery of rules from arbitrary histories.

The paper makes four contributions. First, it frames business and agentic workflows as compositions of decisions that can differ in their need for model reasoning. Second, it defines a trace-guided representation path from English instructions to references or commands, with the accepted English path retained as fallback for current-policy command candidates. Third, it freezes the experimental boundary and requires task-local quality and expense gates before promotion. Fourth, it reports successful, rejected, and nonviable cases across exploratory pilots, confirmatory studies, replications, and a stricter fresh quality-first validation, while separating demonstrated behavior from the broader longitudinal vision.

Material and Methods

Workflow and decision-region model

A business process may contain one or more workflows, and each workflow contains ordered steps $W = (s_1, \dots, s_n)$. PLaND treats the SOP step as its principal unit of analysis, but a step need not be wholly deterministic or wholly model-mediated.

For a semantic step s_i with input region X_i , consider a partition:

$$X_i = X_{D,i} \text{ union } X_{M,i},$$

where $X_{D,i}$ is a validated region that can be resolved by deterministic execution and $X_{M,i}$ is the residual region requiring model-mediated reasoning. The runtime behavior can be represented conceptually as:

$f_i(x) = g_i(x)$, if the deterministic preconditions and guards hold;

$$f_i(x) = M_i(x), \text{ otherwise.}$$

Here g_i is explicit code and M_i is the accepted model-mediated English path. A command can therefore abstain rather than forcing deterministic coverage. This is important for domains in which stable patterns coexist with novel or ambiguous cases.

PLaND supports two practically different forms of compilation. **Operation-level compilation** replaces stable preprocessing, transformation, validation, or tool work with deterministic code while leaving the semantic decision model-mediated. **Decision-region compilation** lets deterministic code itself resolve a validated subset of a semantic decision and sends the residual region to model reasoning. The document-classification schema-v2 pilot is primarily operation-level; the LEDGAR hybrid is primarily decision-region-level.

The broader longitudinal idea is that the validated deterministic region may change as new evidence becomes available. However, the current implementation does not mutate production logic continuously. Each PLaND experiment is bounded by a frozen evaluation contract. A later workflow version should therefore be treated as a new, separately validated evolution cycle rather than an unobserved online rewrite.

PLaND implementation

PLaND uses two Agent Skills. `generate-initial-version` creates a minimal DeepAgent containing exactly one workflow-named SOP skill. It derives task structure from requirements, datasource files, and evaluation schema, but the baseline SOP remains entirely English. Each step receives a stable identifier such as `[S01]` and an English representation marker. The generator explicitly avoids inventing Python or Bash replacements before evaluation evidence exists. Deterministic infrastructure, such as approved datasource tools, may exist from the outset; the English-only claim concerns the SOP representation, not every component of the runtime system.

`pland-evolver` receives the generated agent, labeled development and validation evaluations, a configured runner, a task-local scorer, a quality policy, an optimization metric, and resource limits. Every development case is run from isolated state. The system stores output, trace, latency, tokens, estimated cost, errors, and an immutable snapshot of the SOP.

The evolver is itself an Agent Skill. It instructs the host reasoning/coding agent to cluster development failures and unnecessary expense and propose one bounded change. For